Unidad 1:

INSTALACIÓN, CONFIGURACIÓN Y GESTIÓN DE LOGS EN MYSQL

FICHA TÉCNICA DE LA UNIDAD

Parámetro	Detalle
Unidad Trabajo	UT1: Instalación, Configuración y Gestión de Logs en MySQL 8.x
Resultados de Aprendizaje (RA) Asociados	**RA1:** Identifica los elementos de un sistema gestor de bases de datos analizando sus funciones y relación con el sistema operativo. **RA2:** Instala el sistema gestor de bases de datos especificando las opciones de configuración y describiendo la estructura de sus elementos.
Criterios de Evaluación (CE) Cubiertos	**CE 1.a:** Se han identificado las funciones de los sistemas gestores de bases de datos. **CE 1.b:** Se ha analizado la estructura de almacenamiento de la información. **CE 2.a:** Se han analizado los requisitos del sistema. **CE 2.b:** Se ha seleccionado el motor de almacenamiento. **CE 2.c:** Se han configurado los parámetros del sistema gestor. **CE 2.e:** Se han verificado los ficheros de log del SGBD. **CE 2.f:** Se han resuelto las incidencias de la instalación.
Tiempo	12 horas (Teoría: 2 / Tarea: 4 / Práctica: 6)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. ASPECTOS CRÍTICOS EN LA CONFIGURACIÓN

3. ARQUITECTURA DE ALMACENAMIENTO Y ARCHIVOS DEL MOTOR

4. FICHEROS DE CONFIGURACIÓN

5. GESTIÓN AVANZADA DE VARIABLES

5.1 Ámbitos de Variables: Global vs Sesión

5.2 Variantes Sintácticas de Asignación en MySQL

5.3 SET PERSIST y SET PERSIST_ONLY

6. FUNDAMENTOS TEÓRICOS DE LOS FICHEROS DE REGISTRO (LOGS)

6.1 Clasificación General de las Anotaciones de Bitácora

6.2 Registro de Errores (Error Log) y Filtros JSON

6.3 Registro General de Consultas (General Query Log)

6.4 Registro de Consultas Lentas (Slow Query Log)

6.5 El Registro Binario (Binary Log)

7. DIAGNÓSTICO DE BAJO NIVEL Y AUDITORÍA CON performance_schema

8. BUENAS PRÁCTICAS CON LOS LOGS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

Para un Administrador de Sistemas Informáticos en Red, una base de datos no es simplemente un repositorio de datos relacionales; es un **servicio crítico de infraestructura** que compite de forma directa por los recursos del servidor host (CPU, memoria, almacenamiento e interfaces de red).

La optimización de un SGBD como MySQL requiere entender la relación directa que existe entre el software del gestor y el kernel del sistema operativo. Un mal ajuste de las operaciones de Entrada/Salida (I/O) o una configuración deficiente de los límites de memoria puede provocar problemas catastróficos, desde la ralentización general de las aplicaciones hasta la detención abrupta del servicio debido al Killer de Falta de Memoria del kernel de Linux .

Relación Curricular:

Esta unidad didáctica se conecta con el resto de módulos de ASIR de la siguiente manera:

- **ASO (Administración de Sistemas Operativos):** Gestión de servicios (systemd), permisos a nivel de sistema de archivos, gestión de límites de procesos y almacenamiento persistente en sistemas Linux.
- **SAD (Seguridad y Alta Disponibilidad):** Hardening de red (seguridad de red), cifrado de transmisiones, monitorización activa y análisis forense mediante el parseo automatizado de registros (*logs*).
- **SRI (Servicios de Red e Internet):** Arquitectura cliente-servidor, sockets TCP/IP de escucha, balanceo de puertos y direccionamiento de sockets Unix de dominio local.

2. ASPECTOS CRÍTICOS EN LA CONFIGURACIÓN

La configuración de un motor de bases de datos empresarial es un proceso metodológico que impacta directamente en tres dimensiones: **rendimiento**, **seguridad** y **disponibilidad**. Un SysAdmin debe evaluar rigurosamente los siguientes pilares de configuración al diseñar la infraestructura del servidor:

- **Motor de Base de Datos (Storage Engine):** Elegir el motor de base de datos adecuado es fundamental. La elección depende de las necesidades transaccionales, el volumen de lecturas/escrituras y los requerimientos de integridad referencial. MySQL admite varios motores de almacenamiento, como InnoDB, MyISAM, y más. La elección del motor de almacenamiento depende de tus necesidades específicas. InnoDB es ampliamente utilizado debido a su capacidad de manejar transacciones y su fiabilidad en la recuperación de datos.

- **Modelado de Datos:** Diseñar la estructura física de la base de datos de manera eficiente. Un diseño normalizado evita redundancias y optimiza las rutas de acceso físicas del almacenamiento.

- **Índices:** Configurar índices en las columnas utilizadas frecuentemente bajo cláusulas WHERE, JOIN o ORDER BY. Los índices mejoran drásticamente el rendimiento al acelerar la recuperación de datos, evitando barridos secuenciales completos de tablas (*Full Table Scans*).

- **Ajuste de Parámetros:** Ajustar el archivo de configuración del gestor de acuerdo a las capacidades reales de hardware del host (reparto de RAM para estructuras de cachés, asignación de hilos de CPU y límites de conexiones).

- **Capacidad de Almacenamiento:** Planificar el espacio físico requerido no solo por las tablas del SGBD, sino también por el crecimiento histórico del servicio, archivos temporales, almacenamiento de índices y la acumulación de ficheros de log en disco.

- **Rendimiento:** Supervisar constantemente las métricas de carga del motor para realizar ajustes proactivos y reactivos ante posibles cuellos de botella de hardware o software. Configura adecuadamente los tamaños de búferes como el búfer del sistema de almacenamiento en caché (InnoDB Buffer Pool), el búfer de consulta (Query Cache), y el búfer de índices (Key Buffer), entre otros, para optimizar el rendimiento.

- **Copias de seguridad y recuperación:** Configura rutinas de copia de seguridad y restauración para proteger tus datos contra la pérdida o daño. Define políticas de retención y realiza pruebas de recuperación regularmente. onfigura rutinas de copia de seguridad regulares para proteger tus datos. MySQL ofrece herramientas como mysqldump y utilidades de replicación para este propósito.

- **Seguridad:** Aplica medidas de seguridad, como autenticación, autorización y cifrado de datos, para proteger la base de datos contra amenazas internas y externas. Configura la autenticación y autorización de usuarios de forma

adecuada. Limita el acceso a la base de datos solo a los usuarios necesarios y utiliza contraseñas fuertes. Considera la encriptación de datos en tránsito y en reposo

- **Alta disponibilidad:** Configura la base de datos para garantizar la alta disponibilidad. Esto puede incluir la implementación de réplicas, clústeres o grupos de disponibilidad, según el motor de base de datos que estés utilizando.
- **Monitorización:** Utiliza herramientas de monitorización para supervisar el estado de la base de datos en tiempo real. Esto te permitirá identificar problemas y tomar medidas preventivas. Habilita el registro de MySQL y utiliza herramientas de monitoreo para seguir de cerca el rendimiento y detectar problemas. Puedes utilizar herramientas como MySQL Workbench, phpMyAdmin o soluciones de terceros.
- **Mantenimiento:** Realiza tareas de mantenimiento regular, como la limpieza de registros obsoletos, la reorganización de tablas y la optimización del rendimiento.
- **Escalabilidad:** Diseña la base de datos con la escalabilidad en mente, para que puedas manejar un aumento en la carga de trabajo a medida que tu aplicación crezca. Si es necesario, configura la replicación para mejorar la alta disponibilidad y la escalabilidad. MySQL ofrece opciones como la replicación maestro-esclavo o el grupo de alta disponibilidad (MySQL Group Replication).
- **Cumplimiento normativo:** Si estás manejando datos sensibles o personales, asegúrate de configurar MySQL cumpliendo con las regulaciones de privacidad y seguridad de datos aplicables, como el Reglamento General de Protección de Datos (RGPD) en la Unión Europea.

La configuración de MySQL puede variar según tus necesidades específicas, por lo que es importante entender bien tus requisitos y las características de tu aplicación para realizar una configuración óptima. También es recomendable mantener un registro de la configuración y realizar un seguimiento de los cambios para facilitar la administración y el mantenimiento a lo largo del tiempo.

3. ARQUITECTURA DE ALMACENAMIENTO Y ARCHIVOS DEL MOTOR

En los entornos UNIX, MySQL representa todo recurso como un archivo. Comprender dónde y cómo escribe el SGBD es indispensable para gestionar operaciones de almacenamiento.

- **Data Directory (/var/lib/mysql):** El directorio raíz de los datos. Cada base de datos relacional se mapea directamente con un subdirectorio en esta ruta física.
- **Motores de Almacenamiento (Storage Engines):**
  - **InnoDB:** Motor de almacenamiento predeterminado y el único recomendado para entornos transaccionales de producción. Garantiza

el principio ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Soporta claves foráneas, bloqueos a nivel de registro y recuperación automática tras fallos de energía.

- **MyISAM:** Antiguo motor sin soporte transaccional. Bloquea tablas enteras durante la escritura, lo que genera problemas graves de concurrencia bajo cargas elevadas. Útil únicamente para lecturas ultrarrápidas en datos históricos inmutables.

- **Archivos Clave en InnoDB:**
  - ibdata1: Espacio de tablas del sistema (*system tablespace*). Almacena el diccionario de datos internos de InnoDB y registros de deshacer (*undo logs*).
  - Archivos .ibd: Espacio de tablas independiente por tabla (*file-per-table tablespace*). Cuando la variable innodb_file_per_table está en ON (valor por defecto en MySQL 8), cada tabla de una base de datos almacena sus registros e índices estructurados de forma física en su propio archivo .ibd.
  - ib_logfile0 e ib_logfile1: El registro de rehacer (*Redo Log*). Es una estructura de búfer circular en disco a la que InnoDB escribe inmediatamente las transacciones modificadas en memoria antes de pasarlas a los archivos de datos finales. Esto asegura que, en caso de un corte eléctrico, las transacciones comprometidas se puedan reconstruir al arrancar el servicio.

El tamaño recomendado para optimizar el almacenamiento de InnoDB en servidores dedicados sigue la regla de dimensionamiento de su memoria caché interna (*Buffer Pool*):

$$\text{innodb_buffer_pool_size} \approx \text{Memoria_Total} \times 0.80$$

Para un servidor de base de datos con 16 GB de memoria RAM dedicada, la asignación de caché sería:

$$16\ \text{GB} \times 0.80 = 12.8\ \text{GB}$$

4. FICHEROS DE CONFIGURACIÓN

En una distribución estándar de Linux (como Debian o Ubuntu), la configuración de MySQL está estructurada bajo un modelo de herencia jerárquica modular. El archivo de entrada principal es /etc/mysql/my.cnf. Sin embargo, para mantener el orden, este archivo incluye de forma recursiva todos los subficheros con extensión .cnf alojados en directorios específicos:

```
# Directorio para configuraciones del cliente local (mysql,mysqldump)
/etc/mysql/conf.d/
# Directorio de configuración específico para el servicio demonio del servidor (mysqld)
/etc/mysql/mysql.conf.d/
```

Regla de Precedencia y Sobrescritura:

Si una misma variable se define en múltiples archivos de configuración de MySQL, el motor leerá los archivos de forma secuencial y aplicará siempre **el último valor leído**. El orden secuencial de carga en Linux es:

1. /etc/my.cnf
2. /etc/mysql/my.cnf
3. /usr/etc/my.cnf
4. ~/.my.cnf (Configuraciones de perfil de usuario)

En sistemas Windows: El archivo de configuración se llama my.ini. La ubicación predeterminada puede variar según la versión de MySQL y la forma en que se instaló, pero suele encontrarse en la carpeta de instalación de MySQL, como C:\Program Files\MySQL\MySQL Server x.x

A continuación, se define un archivo de configuración de producción optimizado para el servidor que los alumnos montarán dentro de la carpeta local ./config/custom_mysqld.cnf asociada a su contenedor:

```
[mysqld]
# Red y Puertos
bind-address = 0.0.0.0
port = 3306
mysqlx_port = 33060
# Conexiones y Seguridad
max_connections = 150
autocommit = OFF
default_password_lifetime = 270
character-set-server = utf8mb4
# Memoria de InnoDB
innodb_buffer_pool_size = 900M
innodb_log_file_size = 128M
innodb_file_per_table = ON
# Logs
general_log = ON
general_log_file = /var/log/mysql/query.log
slow_query_log = ON
slow_query_log_file = /var/log/mysql/slow.log
```

```
long_query_time = 2.000000
log_queries_not_using_indexes = ON
```

5. GESTIÓN AVANZADA DE VARIABLES

MySQL contiene cientos de variables dinámicas e inmóviles que dictan su comportamiento. Para consultarlas desde la línea de comandos de manera rápida, un SysAdmin utiliza sentencias SQL basadas en patrones de filtrado:

```
-- Obtener el valor de las conexiones máximas configuradas
SHOW VARIABLES LIKE 'max_connections';
-- Alternativa directa consultando las variables del sistema
SELECT @@global.max_connections;
```

El archivo de configuración de MySQL puede contener una amplia variedad de parámetros que permiten ajustar y personalizar el comportamiento del servidor MySQL. Estos parámetros se dividen en varias categorías, y algunos de los parámetros más comunes incluyen:

- Parámetros de configuración general
  - port: El puerto en el que el servidor MySQL escucha las conexiones.
  - bind-address: La dirección IP en la que el servidor MySQL acepta conexiones. Puedes configurarlo para escuchar en todas las interfaces (0.0.0.0) o una dirección IP específica.
  - socket: La ubicación del archivo de socket Unix para las conexiones locales.
  - basedir: La ubicación base del directorio de instalación de MySQL.
  - datadir: La ubicación del directorio de datos de MySQL.
  - pid-file: La ubicación del archivo que almacena el PID del servidor MySQL.
- Parámetros de seguridad
  - user: El usuario del sistema operativo bajo el cual se ejecutará el servidor MySQL.
  - password: La contraseña de root para el acceso al servidor MySQL. Este parámetro es importante para establecer la contraseña de administrador en la instalación inicial.
  - skip-networking: Deshabilita la comunicación de red, lo que puede aumentar la seguridad en sistemas locales.
- Parámetros de almacenamiento y memoria
  - innodb_buffer_pool_size: Tamaño del búfer del sistema de almacenamiento en caché InnoDB.
  - key_buffer_size: Tamaño del búfer de índices (en motores MyISAM).
  - tmp_table_size y max_heap_table_size: Tamaño de las tablas temporales en memoria.
  - innodb_log_file_size: Tamaño de los archivos de registro de transacciones InnoDB.
- Parámetros de rendimiento
  - query_cache_size: Tamaño del búfer de consulta (Query Cache) que almacena consultas en caché.
  - max_connections: Número máximo de conexiones simultáneas permitidas.
  - thread_cache_size: Número de hilos que se mantienen en caché para conexiones nuevas.

    - innodb_flush_log_at_trx_commit: Controla cuándo se escriben los registros de transacciones en los archivos de registro.
- Parámetros de replicación y alta disponibilidad
    - server-id: Identificador único del servidor en un entorno de replicación.
    - log-bin: Habilita el registro binario para la replicación.
    - binlog_format: Formato del registro binario (por ejemplo, ROW, STATEMENT, MIXED).
    - gtid_mode y enforce_gtid_consistency: Configuración para el uso de Identificadores Globales Únicos (GTID) en la replicación.
- Parámetros de seguridad de autenticación
    - skip-grant-tables: Inicia MySQL sin la autenticación, útil para restablecer contraseñas.
    - validate_password_policy: Política de seguridad de contraseñas (por ejemplo, MEDIUM o STRONG).
    - skip-name-resolve: Evita la resolución inversa de nombres de host para acelerar la autenticación.
- Otros parámetros
    - log-error: Ubicación del archivo de registro de errores.
    - slow_query_log: Habilita el registro de consultas lentas.
    - log_queries_not_using_indexes: Registra consultas que no utilizan índices.
    - character-set-server: Juego de caracteres predeterminado del servidor.

Es importante tener en cuenta que la configuración exacta puede variar según la versión de MySQL y las necesidades específicas de tu aplicación. Los parámetros se pueden agregar, modificar o eliminar según tus requisitos. Además, se recomienda consultar la documentación oficial de MySQL y la referencia de configuración para obtener detalles específicos sobre cada parámetro y sus valores posibles.

5.1 Ámbitos de Variables: Global vs Sesión

Es fundamental que el SysAdmin diferencie entre los dos entornos lógicos en los que actúa una variable dentro de la memoria RAM de la instancia activa:

1. **Global (GLOBAL):** Afecta de manera general al SGBD y a todas las conexiones entrantes nuevas que se establezcan tras el cambio.

2. **Sesión (SESSION o LOCAL):** Afecta única y exclusivamente a la conexión activa del cliente que realiza la modificación. No altera a los demás usuarios conectados ni a los nuevos hilos de comunicación de red.

Las asignaciones dinámicas en caliente residen únicamente en la memoria RAM volátil. Si el servicio de base de datos se reinicia de imprevisto, o el host experimenta un apagado físico, todos estos cambios dinámicos se perderán, volviendo a leer los parámetros estáticos de los ficheros físicos del disco (/etc/mysql/my.cnf).

5.2 Variantes Sintácticas de Asignación en MySQL

El lenguaje SQL de MySQL permite asignar variables de sistema dinámicas a través de diferentes construcciones sintácticas equivalentes. El SysAdmin debe conocerlas para interpretar y depurar scripts legados de administración:

Sintaxis de Variables Globales (GLOBAL):

```
-- Forma 1: Directa mediante modificador de palabra clave
SET GLOBAL max_connections = 1000;

-- Forma 2: Uso del calificador de ámbito con doble arroba (@@)
SET @@GLOBAL.max_connections = 1000;
```

Sintaxis de Variables de Sesión (SESSION o LOCAL):

```
-- Forma 1: Asignación explícita mediante SESSION
SET SESSION sql_mode = 'TRADITIONAL';

-- Forma 2: Uso del modificador local compatible con ANSI
SET LOCAL sql_mode = 'TRADITIONAL';

-- Forma 3: Sintaxis abreviada con calificador doble arroba
(@@SESSION)
SET @@SESSION.sql_mode = 'TRADITIONAL';

-- Forma 4: Sintaxis abreviada con calificador de variables locales
(@@LOCAL)
SET @@LOCAL.sql_mode = 'TRADITIONAL';

-- Forma 5: Calificador genérico simplificado (asume por defecto
@@session)
SET @@sql_mode = 'TRADITIONAL';

-- Forma 6: Asignación implícita de sesión
SET sql_mode = 'TRADITIONAL';
```

5.3 SET PERSIST y SET PERSIST_ONLY

Tradicionalmente, para fijar un cambio de forma definitiva, el SysAdmin debía conectarse al servidor por SSH, editar manualmente el fichero de configuración de MySQL con permisos de superusuario (sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf) y reiniciar el demonio.

MySQL 8.x resuelve este punto de fricción introduciendo la **persistencia SQL nativa sin necesidad de acceder al Shell del sistema operativo**.

```
-- Persistir un cambio en caliente en la memoria del gestor y escribirlo en disco d
e forma concurrente
SET PERSIST max_connections = 250;
```

```
-- Sintaxis alternativa equivalente
SET @@PERSIST.max_connections = 250;
```

¿Cómo funciona internamente?

Al ejecutar la instrucción SET PERSIST, MySQL realiza dos acciones automáticas en un solo paso:

1. Aplica el cambio en tiempo de ejecución en la memoria RAM (SET GLOBAL).
2. Genera o modifica un fichero físico en formato JSON ubicado en el data directory de la base de datos denominado /var/lib/mysql/mysqld-auto.cnf.

```
{
"Version": 2,
"mysql_server": {
    "max_connections": {
        "Value": "250",
        "Metadata": {
            "Timestamp": 1778235720000000,
            "User": "root",
            "Host": "localhost"
        }
    }
}
}
```

Al arrancar, MySQL carga primero el archivo /etc/mysql/my.cnf tradicional y, como última aetpa del proceso de inicialización, lee el archivo JSON /var/lib/mysql/mysqld-auto.cnf para **sobrescribir todos los parámetros estáticos**.

El comando restrictivo SET PERSIST_ONLY:

Existen parámetros que son inalterables en RAM con el motor activo y requieren obligatoriamente un reinicio físico del servidor de bases de datos para su carga (por ejemplo, el puerto de escucha port o la asignación de memoria estática innodb_buffer_pool_size). Intentar usar SET GLOBAL sobre ellos lanzará un error crítico de ejecución.

Para estos casos, utilizamos:

```
-- Escribe la variable en el archivo de autoconfiguración persistente de disco pero
NO intenta aplicarla en caliente en RAM
SET PERSIST_ONLY innodb_buffer_pool_size = 1073741824;
-- Escribe la variable en el archivo de autoconfiguración persistente de disco pero
```

```
NO intenta aplicarla en caliente en RAM
SET PERSIST_ONLY innodb_buffer_pool_size = 1073741824;
```

El cambio tomará efectos reales en el siguiente reinicio del servicio.

Comando para limpiar y revocar la persistencia:

```
-- Eliminar la persistencia de una variable concreta (volverá a heredar el valor de
l fichero my.cnf en el próximo reinicio)
RESET PERSIST max_connections;
-- Limpiar completamente todas las variables añadidas dinámicamente al fichero mysq
ld-auto.cnf
RESET PERSIST;
```

6. FUNDAMENTOS TEÓRICOS DE LOS FICHEROS DE REGISTRO (LOGS)

Un fichero de registro, LOG o bitácora, almacena información cronológica sobre los numerosos eventos producidos en un SGBD, tanto en su instalación, como en su explotación diaria y administración avanzada. Estos ficheros son **esenciales para conocer el comportamiento real del SGBD**, ya que ayudan al administrador a averiguar las causas de un mal funcionamiento y a auditar todas las operaciones que los clientes realizan en el gestor.

6.1 Clasificación General de las Anotaciones de Bitácora

Cualquier flujo de información dirigido a un fichero de logs supone un apunte de información por parte del programa o proceso que se está ejecutando. Generalmente, en un sistema de base de datos profesional, se pueden distinguir tres tipos de anotaciones:

1. **Información de Eventos y Alertas:** Acciones de sistema llevadas a cabo sobre o desde la instancia de base de datos (arranque del servicio, parada controlada, alertas críticas de desbordamiento de RAM o hilos bloqueados).

2. **Estado General de Parámetros de la Instancia:** Consejos prácticos de actuación proporcionados por el propio gestor en caso de que alguna variable interna sobrepase o no un determinado umbral de funcionamiento programado en el software.

3. **Transacciones y Operaciones (Mutación de Datos):** Registro físico de las sentencias ejecutadas por los usuarios que afecten de forma directa a la estructura física de la base de datos (sentencias DDL como CREATE o ALTER) o a la información contenida en ella (sentencias DML como INSERT, UPDATE o DELETE).

6.2 Registro de Errores (Error Log) y Filtros JSON

Este registro almacena información esencial sobre fallos del motor de almacenamiento, violaciones de acceso de red, interrupciones de concurrencia y advertencias en el arranque de la instancia.

Hardening: Registro en Formato JSON

A partir de MySQL 8.0, un administrador de sistemas puede estructurar el log de errores utilizando formato JSON para su posterior parseo y visualización en paneles externos.

Para activarlo en caliente, debemos cargar el componente filter correspondiente del motor:

```
-- Instalar el cargador de logs JSON
INSTALL COMPONENT 'file://component_log_sink_json';
-- Configurar los logs de errores para que escriban tanto en texto tradicional como
en formato estructurado JSON
SET PERSIST log_error_services = 'log_filter_internal;
log_sink_internal; log_sink_json';
```

Esto creará un archivo con formato estructurado denominado error.json dentro del directorio asignado de datos.

6.3 Registro General de Consultas (General Query Log)

Es una bitácora exhaustiva que registra el momento exacto en el que cualquier cliente se conecta o desconecta del servidor de bases de datos, junto con **todas las sentencias SQL** enviadas al motor por el cliente.

```
# Configuración física requerida en /etc/mysql/mysql.conf.d/mysqld.cnf
general_log = ON
general_log_file = /var/log/mysql/query.log
```

Advertencia Crítica de Seguridad y Rendimiento en Producción:

- **Degradación del subsistema de I/O (Rendimiento):** Cada comando que ejecuta un cliente debe escribirse en un archivo físico en disco de manera síncrona antes de responder al usuario. Bajo cargas concurrentes elevadas de miles de clientes por segundo, esto destruye la velocidad de entrada/salida (*I/O bottlenecks*), reduciendo drásticamente el rendimiento del sistema de almacenamiento del servidor.

- **Privacidad e Información Sensible (Seguridad):** El General Log escribe las sentencias en formato plano ASCII. Si un script o sistema web ejecuta comandos de creación de usuarios o inserción de datos (INSERT INTO usuarios (usuario, password) VALUES ('sysadmin', 'SuperSecPass123')), las claves de usuario se escribirán en texto plano dentro de /var/log/mysql/query.log. Si un intruso consiguiera acceso de lectura al servidor de ficheros del sistema operativo, toda la seguridad del sistema se vería comprometida.
- **Consumo Masivo de Disco (Almacenamiento):** En bases de datos con millones de operaciones diarias, este log puede agotar en cuestión de horas el espacio total del disco duro si no se parametriza una correcta rotación y mantenimiento de espacio físico.

Uso Recomendado: Activar únicamente durante ventanas controladas de depuración de software en entornos locales de desarrollo, o bajo estrictas auditorías de seguridad a nivel de red aislada. Desactivar inmediatamente al finalizar las pruebas.

6.4 Registro de Consultas Lentas (Slow Query Log)

La herramienta primordial de optimización que tiene un SysAdmin. Registra aquellas sentencias SQL cuya ejecución interna en CPU supera el umbral de tiempo especificado en milisegundos en la variable de sistema long_query_time.

Configuración e Implementación:

```
-- Activar el motor de registro de consultas lentas a nivel global y persistente
SET PERSIST slow_query_log = ON;
SET PERSIST slow_query_log_file = '/var/log/mysql/slow.log';
-- Fijar el umbral de tiempo máximo a 2 segundos exactos
SET PERSIST long_query_time = 2.000000;
-- Capturar también todas aquellas consultas ineficientes que no utilizan índices p
ara su ejecución
SET PERSIST log_queries_not_using_indexes = ON;
```

Generación de Prueba y Simulación de Latencia:

Para comprobar que el registro de consultas lentas funciona correctamente, el administrador simula una parada del hilo de ejecución mediante la función de pausa de MySQL:

```
-- Provocar de forma artificial una ejecución lenta controlada de 3 segundos
SELECT NOW(), SLEEP(3);
```

Parseo Técnico del Registro:

Al abrir el fichero /var/log/mysql/slow.log en el host o contenedor, el administrador observará la siguiente estructura detallada:

```
# Time: 2026-06-07T11:42:01.354210Z
# User@Host: root[root] @ localhost [] Id: 12
# Query_time: 3.001240 Lock_time: 0.000000 Rows_sent: 1
Rows_examined: 1
SET timestamp=1778235721;
SELECT NOW(), SLEEP(3);
# Time: 2026-06-07T11:42:01.354210Z
# User@Host: root[root] @ localhost [] Id: 12
# Query_time: 3.001240 Lock_time: 0.000000 Rows_sent: 1 Rows_examined:1
SET timestamp=1778235721;
SELECT NOW(), SLEEP(3);
```

Métricas Clave de Análisis:

- Query_time: El tiempo que la consulta bloqueó la CPU del gestor de base de datos (3.001240 segundos).
- Lock_time: El tiempo que la sentencia tuvo que esperar en cola debido a un bloqueo de tabla o fila por otra transacción concurrente.
- Rows_sent: Cantidad de filas devueltas al cliente.
- Rows_examined: El número real de registros que el motor tuvo que buscar en el almacenamiento físico en disco antes de poder responder. Si Rows_examined es muy elevado en comparación con Rows_sent, significa que la consulta no cuenta con un índice B-Tree óptimo y obliga al motor a realizar un escaneo completo de la tabla (*Full Table Scan*).

6.5 El Registro Binario (Binary Log)

El log binario (binlog) es un registro estructurado que contiene todos los eventos que describen cambios en la base de datos, como modificaciones de tablas (DDL) o modificaciones en los datos de las tablas (DML). Se almacena en formato binario para garantizar la velocidad de escritura.

Propósitos Fundamentales:

1. **Replicación de Datos:** En un clúster MySQL (Maestro-Réplica), el servidor principal (*Master*) envía las instrucciones escritas en su log binario al servidor secundario (*Replica*) para que este ejecute las transacciones locales y mantenga el estado de los datos sincronizado.
2. **Operaciones de Recuperación tras Fallos (Recovery):** Permite recuperar el estado de una base de datos después de restaurar una copia de seguridad en frío. Al ejecutar el binlog a partir de la fecha de la copia, el motor re-aplica de forma exacta todas las operaciones

transaccionales comprometidas hasta el instante del desastre (*Point-in-time recovery*).

7. DIAGNÓSTICO DE BAJO NIVEL Y AUDITORÍA CON performance_schema

El performance_schema es una potente base de datos interna de bajo nivel de MySQL encargada de recopilar estadísticas operativas en tiempo real de la instancia con una sobrecarga de recursos casi nula.

Cuando se producen errores recurrentes de conexión de usuarios o problemas de autenticación, el SysAdmin puede analizar de forma detallada las tablas internas de rendimiento.

Consulta para auditar incidencias y clasificar errores por usuario:

La tabla events_errors_summary_by_user_by_error monitoriza de forma centralizada todos los códigos de error disparados por los distintos perfiles de usuario. La siguiente consulta devuelve un desglose de los últimos 20 errores registrados para detectar posibles ataques de fuerza bruta o desajustes de conexión:

```
SELECT USER,ERROR_NUMBER, ERROR_NAME, COUNT_STAR AS TOTAL_OCURRENCIAS, FIRST_SEEN A
S PRIMERA_VEZ, LAST_SEEN AS ULTIMA_VEZ
FROM performance_schema.events_errors_summary_by_user_by_error
WHERE USER IS NOT NULL AND COUNT_STAR > 0
ORDER BY LAST_SEEN DESC
LIMIT 20;
```

Comprobación de Actividad SQL y Contador de consultas realizadas:

La tabla status_by_user muestra las variables de estado acumuladas por hilos de conexión individuales. Para obtener una visión cuantitativa de cuántas lecturas de datos (SELECT) ha realizado el usuario root para evaluar la carga de trabajo en el servidor:

```
SELECT USER, VARIABLE_NAME, VARIABLE_VALUE
FROM performance_schema.status_by_user
WHERE USER = 'root' AND VARIABLE_NAME LIKE 'Handler_read%'
ORDER BY CAST(VARIABLE_VALUE AS UNSIGNED) DESC;
```

8. BUENAS PRÁCTICAS CON LOS LOGS

Un servidor de bases de datos de nivel profesional debe garantizar la protección, el aislamiento y la autolimpiabilidad de sus archivos físicos.

1. Aislamiento de Permisos en el Sistema de Archivos de Linux:

Los archivos de logs contienen metadatos y en ocasiones credenciales que nunca deben ser expuestos a usuarios locales del sistema operativo que no pertenezcan al grupo de administración de la base de datos.

```
# Asignar la propiedad al usuario y grupo del demonio mysqld
sudo chown -R mysql:mysql /var/log/mysql
# Restringir permisos de lectura/escritura únicamente al propietario
sudo chmod 750 /var/log/mysql
sudo chmod 640 /var/log/mysql/*.log
```

2. Rotación de Logs para Evitar que el Disco se Llene:

Un query.log mal gestionado en producción puede acumular cientos de gigabytes en cuestión de días y provocar un fallo total del sistema de archivos al agotar los inodos o el espacio de almacenamiento disponible del servidor físico.

Para evitarlo, se configura el servicio logrotate en Linux agregando un archivo de configuración específico:

```
# Archivo de Configuración del Rotador de Logs del Sistema Linux: /etc/logrotate.d/
mysql-server
/var/log/mysql/*.log {
    weekly
    rotate 4
    size 100M
    compress
    delaycompress
    missingok
    notifempty
    sharedscripts
    postrotate
    # Forzar a MySQL a reabrir y recrear los archivos físicos tras la rotación de l
ogs sin reiniciar el servicio
        /usr/bin/mysqladmin flush-logs
    endscript
}
```